

Sincronizar schema: status de entrega

Data 2026-09-02 Branch feature-dev-breno-automacao Commit base e2dcc57

PREMISSA A tarefa fala em migrations 004 a 007. No repositório existem dez, de 001 a 010: as 008, 009 e 010 entraram no commit e2dcc57, depois que a tarefa foi criada. A 010 cria o bucket de storage e as policies das quais o workflow n8n de ingestão via WhatsApp depende, então deixar essas três fora da conferência era risco real. Tudo abaixo cobre 004 a 010.

As cinco atividades

#	ATIVIDADE	STATUS	RESULTADO
1	Rodar as migrations pendentes em gc-dev	CONCLUÍDA	Não havia pendência. Verificado por catálogo: 21 de 21 checagens OK, 004 a 010 aplicadas
2	Aplicar as mesmas migrations em gc-prod	EM ESPERA	gc-prod está pausado. Procedimento pronto, com backup e janela, para quando voltar
3	<code>npm run db:types</code> e conferir <code>types.ts</code>	CONCLUÍDA	2267 para 2449 linhas, com os três objetos que faltavam
4	<code>npx tsc --noEmit</code> sem erro	CONCLUÍDA	Passa limpo, e agora o teste realmente pode falhar
5	<code>npm run lint</code> sem warnings	CONCLUÍDA	Sem warnings nem erros

1 Migrations em gc-dev

Concluída. Não havia pendência.

O repositório não tinha controle de migrations, sem `supabase/migrations` e sem CLI, então não existia histórico para consultar. Foi escrito o `supabase/verificar_schema_aplicado.sql`, uma query somente leitura sobre os catálogos do Postgres que devolve uma linha por evidência. Resultado em gc-dev: **21 de 21 checagens OK.**

MIGRATION	CHECAGENS	EVIDÊNCIA NO CATÁLOGO
004	8 de 8	tabela <code>clientes</code> , <code>obras.cliente_id</code> NOT NULL, FK <code>obras_cliente_fk</code> , zero colunas legacy em <code>obras</code> e <code>orcamentos</code> , rename para <code>observacao</code> , 4 views recriadas
005	2 de 2	RLS ativo e 4 policies em <code>clientes</code>
006	1 de 1	<code>calcular_valoresobra</code> com o CASO 2 (<code>fonte=sem_proposta</code>)
007	1 de 1	<code>progresso_itens_pct</code> exposto em <code>obras_com_valores</code>
008	4 de 4	tabela <code>documentos_processamento</code> , RLS, 4 policies, 3 índices
009	3 de 3	tabela <code>contatos_whatsapp</code> , RLS, 4 policies
010	2 de 2	bucket privado de 20 MB restrito a <code>application/pdf</code> e 4 policies em <code>storage.objects</code>

Contexto do catálogo: 16 tabelas e 53 policies em `public`, coerente com o schema completo.

2 Migrations em gc-prod

Não executada. gc-prod está pausado.

Fica registrada com o procedimento pronto. Quando prod voltar: rodar o script de verificação, comparar com a saída de dev, fazer backup, abrir janela de manutenção e aplicar o que faltar na ordem 004 até 010.

O backup não é formalidade. A 004 remove treze colunas de `obras` e seis de `orcamentos` depois de copiar os dados para `clientes`, sem `if exists`, e é irreversível. A ordem também importa: a 004 recria `obras_com_valores` no final, sem a coluna `progresso_itens_pct` que a 007 adiciona, e a listagem de obras seleciona essa coluna por nome. Aplicar a 004 depois da 007 quebra a página em runtime, não no build.

3 Types regenerados

Concluída.

O `src/lib/supabase/types.ts` estava três objetos atrás do schema. Depois de regenerar, passou de 2267 para 2449 linhas, com 192 linhas adicionadas e 10 removidas. Entraram:

1. `contatos_whatsapp`, 8 colunas
2. `documentos_processamento`, 13 colunas

3. `progresso_itens_pct` em `obras_com_valores`

As 10 remoções são reformatação do gerador nos helpers genéricos, nenhum objeto perdido. A geração também respondeu uma dúvida aberta: o project id `gzbmhgnpoehornmidmagg`, fixo no script, é o `gc-dev`.

Dois problemas apareceram no caminho e foram corrigidos:

1. O comando falhou por falta de access token, e o redirecionamento com `>` truncou o `types.ts` para 0 bytes antes de a CLI rodar. O arquivo foi restaurado do commit `e2dcc57`, idêntico ao original. O script agora gera num arquivo temporário e só promove se a geração terminar bem, então uma falha deixa o arquivo bom intacto. Ganhou também suporte a `$SUPABASE_PROJECT_ID` e um `db:types:dev` ao lado.
2. O login por navegador da CLI não funciona sem TTY. Foi usado personal access token via variável de ambiente.

4 tsc e o teste que não podia falhar

Concluída, e o teste passou a valer algo.

Passa limpo, no worktree e no checkout principal. Vale registrar que ele já passava antes, com os types velhos, porque as três páginas de listagem usavam `as unknown as`, que descarta a comparação entre a string do `select()` e o `Database` gerado. Era um teste que não podia falhar.

Trocado por um `as` só nas três (`obras`, `fd`, `orcamentos`), o que mantém o narrowing dos campos NOT NULL e devolve a checagem ao compilador. Verificado injetando uma coluna inexistente no `select()`:

```
error TS2352: Conversion of type
'SelectQueryError<"column 'coluna_que_ nao_existe' does not exist
on 'obras_com_valores'.">[]' to type 'ObraListItem[]' may be a
mistake
```

Antes isso passava batido.

5 lint

Concluída.

`npm run lint` retorna sem warnings nem erros.

Feito além do checklist

1. As 008 e 009 rodavam fora de transação, então uma falha no meio deixaria a tabela criada sem polícies. Foram envolvidas em `begin;`

```
commit; .
```

2. O repositório passou a usar `supabase/migrations` com a CLI. As dez migrations viraram arquivos com timestamp, usando as datas reais dos commits que introduziram cada uma, então a ordem cronológica é a mesma da numeração antiga. O de para está no `supabase/README.md`. Criado também o `config.toml`, validado pela própria CLI. Os scripts que não são migrations (`000_reset_dev`, `setup_inicial_dev`, `seed_orcamentos` e o de verificação) ficaram fora de `migrations/` e nunca entram num `db push`.

Pendências

ATENÇÃO O `.env.local.example` é rastreado pelo git e está com a URL do gc-dev, uma publishable key e um personal access token na cópia de trabalho. No commit e2dcc57 ele está em branco, então nada foi para o histórico ainda, mas um `git add -A` levaria o segredo junto. Correção: `cp .env.local.example .env.local` e depois `git checkout -- .env.local.example`. Atenção ao nome da variável: o código lê `NEXT_PUBLIC_SUPABASE_ANON_KEY`, não `PUBLISHABLE_KEY`.

1. Rodar `supabase migration repair --status applied` nas dez versões. As dez já estão aplicadas em dev, mas o histórico remoto da CLI está vazio. Um `db push` antes do repair tentaria reaplicar tudo a partir da 004, que é destrutiva. Ela aborta sozinha por estar em transação, mas o susto é evitável. Comandos exatos no `supabase/README.md`.
2. Tirar as credenciais do `.env.local.example`, conforme o aviso acima.
3. Commitar. As alterações estão espelhadas no checkout principal, ainda sem commit. O histórico detalhado está na branch `worktree-verificar-schema-4-1`, que parte de e2dcc57 e entra como fast forward.
4. `gc-prod`, quando despausar, conforme a atividade 2.

Arquivos

ARQUIVO	O QUE É
supabase/verificar_schema_aplicado.sql	verificação somente leitura de 004 a 010, segura em produção
supabase/migrations/	as dez migrations sob o histórico da CLI
supabase/config.toml e supabase/README.md	config mínima e runbook de adoção
docs/4.1-sincronizar-schema-analise.md	análise completa, com o diagnóstico e o raciocínio

Estado de gc-dev verificado no catálogo em 2026-09-02. gc-prod pausado.